

Stack Técnico Único + CarrilPath de Aplicación y Aprendizaje

Sistema Integral de Panadería Multi-Sucursal

Propuesta ejecutiva y plan de implementación (16 semanas)

23 de agosto de 2025

Resumen

Este documento define un **stack único, concreto y moderno** orientado a dos objetivos simultáneos: (1) construir un sistema funcional y escalable para la panadería, con buenas prácticas de ingeniería, y (2) desarrollar las competencias que hoy piden los puestos de *Data Scientist / Data & ML Engineer*. Se incluye un **CarrilPath de 16 semanas** con entregables, definición de terminado (DoD), artefactos de portafolio y mapeo explícito a habilidades de mercado.

1. Stack técnico único (sin alternativas)

1.1. Núcleo de datos

- **OLTP:** PostgreSQL 16/17.
- **Time-series:** TimescaleDB (extensión sobre PostgreSQL) para telemetría/sensores.
- **ORM & migraciones:** SQLAlchemy 2.x + Alembic.

1.2. Backend (API + dominio)

- **Framework:** FastAPI (Python 3.12+), Pydantic, OpenAPI autogenerado.
- **Background jobs:** Celery + Redis.
- **Tiempo real:** WebSockets en FastAPI.

1.3. Frontend (POS, Admin, Reportes operativos)

- **Web:** Next.js (App Router) + React + TypeScript.
- **UI:** Tailwind CSS + Radix UI.
- **Estado remoto:** TanStack Query.
- **Formularios/validación:** React Hook Form + Zod.
- **PWA offline (POS):** Workbox (colas offline y sincronización).

1.4. Plataforma de datos, analítica y ML

- **Orquestación:** Apache Airflow 2.
- **Transformaciones SQL (ELT):** dbt Core.
- **Experiment tracking y registry:** MLflow.
- **Calidad de datos:** Great Expectations.
- **Modelado:** scikit-learn, XGBoost/LightGBM, statsforecast.
- **Exploración local:** DuckDB (notebooks/analítica ad-hoc).

1.5. Observabilidad y confiabilidad

- **Trazas/métricas/logs:** OpenTelemetry + OTEL Collector → Prometheus & Grafana.
- **Errores/perf:** Sentry (frontend y backend).

1.6. Seguridad, identidad y cumplimiento

- **IdP:** Keycloak (OIDC/OAuth2) + roles por sucursal/perfil (NextAuth en frontend).
- **Secretos:** HashiCorp Vault.
- **Hardening:** OWASP ASVS, Argon2, 2FA en backoffice.

1.7. DevEx, DevOps e infraestructura

- **Deps:** Poetry (Python), pnpm (frontend).
- **Calidad:** ruff + black (Py), eslint + prettier (TS), pre-commit.
- **Contenedores:** Docker; Docker Compose (dev).
- **CI/CD:** GitHub Actions (lint, tests, build, migraciones, deploy).
- **Prod:** Kubernetes (k3s/EKS/GKE) + Helm + Traefik (Ingress).
- **IaC:** Terraform (red, DB administrada, buckets).
- **Backups/DR:** PITR de PostgreSQL; retenciones por entorno.

1.8. BI y autoservicio

- **BI interno:** Metabase (KPIs: ventas, top productos, mermas, etc.).
- **Catálogo/linaje:** dbt docs (publicado en intranet).

2. Arquitectura de repos y ambientes

2.1. Monorepo recomendado

Listing 1: Estructura de carpetas (monorepo)

```
/zero-merma
/app
  /backend      # FastAPI + domain + Alembic + Celery
  /frontend     # Next.js (POS/admin), Tailwind, RHF, Zod
  /shared       # Schemas, tipos, contratos OpenAPI
/data
  /warehouse    # dbt, seeds, macros, tests
  /airflow      # DAGs ETL/ML, operadores
  /ml           # notebooks, MLflow client, pipelines
  /ge           # Great Expectations (expectations, suites)
/infra
  /k8s          # charts Helm por servicio
  /terraform    # m dulos de red, DB, buckets, CDNs
  /observability # OTEL, Prometheus, Grafana, Sentry
/ci
  github-actions/ # workflows .yaml
/docs
  architecture, ADRs, dbt docs export, runbooks
```

2.2. Ambientes

- **dev:** desarrollo local + Docker Compose (semillas de datos sintéticos).
- **stg:** entorno de *staging* para pruebas de integración y UAT.
- **prod:** Kubernetes gestionado, PostgreSQL administrado (PITR), monitorizado.

3. CarrilPath (16 semanas): entrega de producto + aprendizaje DS

3.1. Filosofía

Cada bloque semanal incluye: **Objetivo de producto**, **Entregables (DoD)**, **Artefactos de portafolio** y **Competencias DS/ML**. El orden prioriza valor operacional temprano (ventas/inventario) sin perder el tren de datos (lineaje, calidad, orquestación).

Semana	(Foco)	DoD (Entregables verificados)	Competencias DS/ML
1	(Setup)	Infra dev con Docker Compose; repos iniciales; FastAPI & Next.js & Postgres corriendo; Alembic baseline; Keycloak en dev; lint/format + pre-commit; GH Actions con lint + tests.	Buenas prácticas de ingeniería; control de versiones de DB; testing básico; lectura de especificaciones técnicas.
2	(Modelo de dominio)	Módulos <i>users/branches/products/recipes</i> ; FKs, seeds mínimas; endpoints CRUD tipados; contratos OpenAPI; Storybook/Radix para UI base.	Modelado relacional; esquemas tipados; diseño de APIs; contratos cliente-servidor.
3	(Inventario (ledger))	Tabla de movimientos e invariantes; endpoints de entradas/salidas/ajustes; vistas de stock por sucursal; índices clave; pruebas de reconciliación.	Trazabilidad y consistencia; diseño de <i>event sourcing</i> light para stock.
4	(POS (ventas, pagos))	Flujo POS end-to-end (carrito, cobro, emisión de ticket); sesión de caja (única abierta por caja/usuario); pagos múltiples; validaciones y DoD funcional.	Diseño de estados finitos; validación estricta; <i>error budgets</i> operativos.
5	(PWA Offline)	Workbox: colas offline de ventas; sincronización al reconectar; pruebas e2e; métricas de reintentos.	Arquitecturas <i>resilient/offline-first</i> ; telemetría de cliente.
6	(Producción (lotes))	Órdenes de producción, consumo real (detallado), mermas; <i>postings</i> al inventario; reportes de costo por lote.	Cálculo de costos; diseño de <i>flows</i> con efectos laterales controlados.
7	(Compras y proveedores)	Órdenes de compra, recepciones con lotes/caducidad; integración con inventario; <i>stock in transit</i> .	Data modeling con lotes y FEFO; series de tiempo de existencias.
8	(Telemetría de tienda)	TimescaleDB + sensores (lecturas); partición temporal; retención; dashboards operativos (Grafana).	Modelado de series temporales; agregaciones ventana; <i>downsampling</i> .

Semana (Foco)	DoD (Entregables verificados)	Competencias DS/ML
9 (Capa ELT (dbt))	Modelo estrella de ventas/inventario; <i>staging</i> y <i>marts</i> ; tests de singularidad y FKs; documentación generada (dbt docs).	Modelado dimensional; validación de datos; linaje.
10 (Calidad de datos (GE))	Expectations para ventas, ítems, pagos, stock; <i>data docs</i> publicados; <i>checkpoints</i> en CI.	Testing de datos; monitoreo de <i>SLAs/SLIs</i> de datos.
11 (Orquestación (Air-flow))	DAGs diarios: ingest, transformaciones dbt, <i>backfills</i> , reconciliación de inventario; <i>data-aware scheduling</i> .	Diseño de DAGs; <i>idempotency</i> ; <i>backfilling</i> .
12 (Feature store ligero)	Extracción de features (ventas, clima opcional, festivos) y <i>point-in-time correctness</i> ; MLflow tracking.	Prevención de <i>data leakage</i> ; tracking de experimentos.
13 (Forecasting de-manda)	<i>Baseline</i> con <i>statsforecast</i> /LightGBM; <i>cross-validation</i> temporal; backtesting; métricas (sMAPE, MASE).	Series de tiempo; evaluación robusta; selección de modelos.
14 (Serving de pronósticos)	Endpoint de pronóstico por sucursal/producto; <i>model registry</i> (MLflow) con alias PROD; monitoreo de deriva simple.	MLOps: despliegue y versión de modelos; monitoreo en producción.
15 (BI y KPIs)	Metabase: tableros de ventas, rotación, merma, <i>forecast vs actual</i> ; permisos por rol/sucursal.	Storytelling de datos; definición de KPIs; gobierno de acceso.
16 (Endgame & Portafolio)	Hardening (seguridad, backups/DR), carga base; revisión de <i>SLOs</i> ; demo ejecutiva y portafolio DS publicado.	Arquitectura <i>production-grade</i> ; presentación técnica; documentación.

3.2. Artefactos de portafolio por bloque

- **Código y contratos:** repos públicos (o extractos), OpenAPI, ADRs.
- **Data:** *marts* dbt con tests, *data docs*, GE *expectations*.
- **ML:** notebooks de exploración (DuckDB), MLflow runs, *model card*.
- **Ops:** workflows de CI/CD, DAGs de Airflow, *dashboards* Grafana/Metabase.
- **Demo:** video corto (3–5 min) mostrando POS offline, reconciliación, tableros, y forecast en tiempo real.

4. Definición de Terminado (DoD) y *Quality Gates*

4.1. DoD por historia

- Endpoint/documento: OpenAPI actualizado, pruebas unitarias (+80 % en dominio crítico), validaciones exhaustivas.
- Esquema/migración: Alembic *up/down*, índices y FKs, datos semilla.
- Vista UI: estados de carga/error vacíos, accesibilidad básica (navegación por teclado), formularios con Zod.

- Datos: tests dbt (único, no nulo, relaciones) en verde; GE *checkpoint* para tablas críticas.
- ML: experimento trazado en MLflow con métricas, semillas reproducibles y *model card*.

4.2. Quality gates en CI/CD

- Lint/format obligatorio (ruff/black, eslint/prettier).
- Pruebas unitarias + de contrato; *smoke tests* tras despliegue.
- dbt tests + GE *validation* antes de correr pronósticos.
- Escaneo básico OWASP (dependabot, *secrets* escaneados).

5. Plan de estudio paralelo (2h/día, 5 días/semana)

Estructura semanal

1. 45 min: documentación oficial de la herramienta del bloque (*focus of the week*).
2. 45 min: tutorial guiado sobre el *feature* implementado (reproducir en tu código).
3. 30 min: escritura de notas y *cheat-sheets* personales.

Prioriza: Postgres/Timescale, FastAPI/Pydantic, Next.js App Router, dbt, Airflow, MLflow, GE, Prometheus/Grafana.

6. KPIs operativos y de datos

6.1. Operación de tienda

- Venta diaria por sucursal, ticket promedio, top productos, rotación de inventario, merma %.
- Tiempos de cobro, tasa de reintento offline, disponibilidad de POS, discrepancias caja vs sistema.

6.2. Plataforma de datos/ML

- *Freshness* de datos (dbt), éxito de DAGs, latencia de pipelines.
- Calidad (GE): proporción de tests verdes; *data drift* simple.
- Desempeño de forecast: sMAPE/MASE por familia/sucursal; alertas de deriva.

7. Guía mínima de arranque (extracto)

Backend

```
poetry init && poetry add fastapi uvicorn[standard] pydantic-settings
poetry add sqlalchemy alembic psycpg[binary] redis celery
poetry add python-dotenv passlib[bcrypt] itsdangerous
poetry add loguru
```

Frontend

```
pnpm dlx create-next-app@latest frontend --ts --eslint --src-dir --app
--tailwind
pnpm add @radix-ui/react-dialog @tanstack/react-query zod react-hook-
form
pnpm add workbox-window
```

Data/ML

```
# dbt (Postgres), Airflow (con docker-compose), MLflow, GE
pip install dbt-postgres apache-airflow mlflow great_expectations duckdb
```

8. Riesgos y mitigaciones

- **Complejidad inicial:** Monorepo, plantillas de servicio y *scaffolding* para reducir tiempo de arranque.
- **Datos sucios:** GE + dbt tests integrados a CI antes de *servir* pronósticos.
- **Operación offline:** Workbox con *retry queues* y *backoff*.
- **Costos nube:** Entorno de *staging right-sized* y políticas de retención en Timescale.

9. Resultado esperado al finalizar las 16 semanas

- Sistema POS & backoffice estable, offline-first, multi-sucursal.
- *Data platform* con linaje, calidad y orquestación: métricas y tableros vivos.
- Modelo de demanda en producción (registro en MLflow), con monitoreo y *rollbacks* posibles.
- Portafolio con código, documentación y demo ejecutiva, alineado a vacantes de DS/ML.